# Informe técnico integrador — TPI, segunda entrega

**Proyecto:** Food Store
**Cátedra:** Base de Datos II — Tecnicatura Universitaria en Programación — UTN FRM — Ciclo 2026
**Docente:** Sergio Neira
**Integrantes:** Tiziano Valentini y Juan Martin Garcia
**Motores:** PostgreSQL 16+ (mediciones en PostgreSQL 16.15 y 18.3), MongoDB (`mongosh`) y SQLite 3.37+ `STRICT`
**Repositorio:** `Food-Store-Bd-Valentini-Garcia`, rama `main` (`d9bfa6d`), que ya incorpora la rama `trabajos-practicos-nosql`
**Alcance:** Unidades 4 a 8. La capa relacional de las unidades 1 a 3 queda en `TP1_FoodStore` a `TP5_Indices_Vistas` y no se rehace en este informe.

Este informe resume lo actuado. Las salidas completas están en las carpetas de cada unidad. El protocolo de copia de trabajo, transacción reversible y respaldo está en `protocolo_seguridad.md`.

## 1. Qué se implementó en cada unidad

### Unidad 4 — Normalización avanzada y redundancia (`TP6_FNBC_Desnormalizacion`)

La tabla heredada `control_lote_almacen` no está en FNBC: `responsable_control_id → deposito_id` y `deposito_id` es primo. Se descompuso, sin pérdida, en `responsable_deposito` y `asignacion_control_lote`. La vista `vista_control_lote_almacen_fnbc` reconstruye las tres filas de la instancia (501/30/801, 502/30/801, 503/31/802).

La desnormalización controlada agrega solo `detalle_pedido.categoria_nombre`, mantenida por triggers en la misma transacción. La fuente de verdad sigue siendo `categoria.nombre_categoria`. El plan de reversión está en `herramientas/revertir_desnormalizacion.sql`. La especificación EARS está en `kiro/requirements.md` y `kiro/design.md`.

### Unidad 5 — Administración y seguridad (`TP7_Seguridad_Roles_Auditoria`)

Se implementó un esquema de mínimo privilegio con seis roles de grupo (`rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes`, `rol_logistica`, `rol_auditor`) y dos roles de login (`app_web`, `admin_datos`). `rol_soporte` y `rol_auditor` leen de `usuario` solo `id, nombre, email, rol, fecha_alta`. `password_hash` se toca únicamente desde `fn_autenticar` y `fn_resetear_password`, ambas `SECURITY DEFINER` con `search_path` fijo. `rol_logistica` puede actualizar `stock` y `activo`, no `precio_actual`. Las contraseñas de los roles de login no están en `sql/roles.sql`: entran por variables de `psql`. La spec Kiro está en `docs/kiro_roles_spec/`.

La auditoría nativa (`log_connections`, `log_disconnections`, `log_statement`) se probó con una conexión real de `app_web`. El simulacro de acceso indebido está en `sql/simulacro_incidente.sql`. La anonimización determinista de `usuario_anon` está en `sql/usuario_anon.sql`.

### Unidad 6 — Mantenimiento (`TP8_Mantenimiento_Respaldo`)

El plan operativo está en `SPEC_U6.md`. `backup_foodstore.ps1` toma un `pg_dump -Fc` de `foodstore`. `health_check.sql` y `maintenance.sql` cubren el control diario y el `VACUUM (VERBOSE, ANALYZE, INDEX_CLEANUP ON)`. `restore_test.ps1` restaura contra reloj en la base descartable `foodstore_restore_test`, sin reemplazar `foodstore`. El informe post-incidente es `informe_u6.md`.

### Unidad 7 — NoSQL (`TP7_NoSQL_MongoDB`)

Se modeló el pedido como documento agregado en la colección `pedidos` de `food_store_nosql`: cliente embebido y arreglo `lineas`. La colección se crea con validador `$jsonSchema` (`validationLevel: strict`, `validationAction: error`). Los importes van en `Decimal128` (`NumberDecimal`), no en `double`. Se migraron 6 pedidos reales desde PostgreSQL. Las consultas documentales y las equivalentes relacionales están en `consultas_mongodb.js` y `consultas_postgresql.sql`.

### Unidad 8 — Base móvil e integración (`TP8_Movil_SQLite_Cierre`)

La tablet usa SQLite `STRICT` con `PRAGMA foreign_keys = ON`. El esquema (`producto_cache`, `pedido_borrador`, `item_borrador`, `cola_sync`, `snapshot_servidor`) y la sincronización están especificados en `docs/kiro/` y generados en `movil/schema_sqlite.sql`, `movil/semaforo.sql` y `movil/simulador.py`. La regla es asimétrica: el móvil propone borradores y el servidor dispone. `SIMULADO_APROBADO` no confirma una venta ni descuenta stock en PostgreSQL.

La instantánea sale de `v_snapshot_movil` (`movil/vista_postgres_snapshot.sql`) y queda en `movil/servidor_snapshot.csv` (10 productos reales). El code review final de los scripts de la cursada está en la sección 3.c de `docs/informe.md`, con la decisión humana sobre cada hallazgo.

## 2. Cómo se probó

Todas las pruebas de PostgreSQL corrieron sobre copia de trabajo (`foodstore`, `food_store_dev` o `foodstore_restore_test`), nunca reemplazando la base de origen durante un ensayo destructivo.

**Unidad 4.** `tp_fnbc_control_lote.sql` corre la migración y las anomalías del esquema viejo dentro de transacciones que terminan en `ROLLBACK` si el `EXCEPT` no queda vacío. `herramientas/pruebas_integridad.sql` ejercita inserción, corrección de un valor falso, update directo de la columna redundante, cambio de producto, reclasificación, renombre de categoría, bajas lógicas y `DELETE` en cascada; cada caso se revierte. La consulta del top de categorías se midió con `EXPLAIN (ANALYZE, BUFFERS)` antes y después, siete veces, sobre 200.003 pedidos y 400.011 detalles. La reversión de la columna se verificó con `revertir_desnormalizacion.sql`.

**Unidad 5.** `roles.sql` se aplicó en la copia y se contrastó con `\du` y `role_table_grants`. La auditoría se activó con `ALTER ROLE app_web SET log_statement = 'all'` y una conexión real `\connect - app_web`, porque `SET ROLE` no dispara `log_connections` ni deja cambiar `log_statement` a un rol sin superusuario. El simulacro (PID 12492) hizo cinco autenticaciones fallidas, un login exitoso, `SELECT *` sobre `usuario`, lectura de las columnas permitidas, `SET ROLE admin_datos`, dos `GRANT` y un `DELETE`. Las sentencias fallidas se dejaron salir al motor (`ON_ERROR_STOP off`) para que el log conserve el `ERROR`. Antes de enviar el log a la IA se anonimizaron correos, contraseñas, IP y rutas. La equivalencia de `usuario_anon` se probó con `EXCEPT ALL` en ambos sentidos.

**Unidad 6.** Se generó `backups/foodstore_20261109_221751.dump` y, 1,58 s después, `restore_test.ps1` cronometró `createdb`, `pg_restore --no-owner --no-privileges` y tres validaciones SQL contra `foodstore`. El binario del dump no viaja en Git (`*.dump` está en `.gitignore`); quedan el log, el tamaño, el SHA-256 truncado y el reporte del simulacro.

**Unidad 7.** En `mongosh` se insertaron los 6 pedidos con `Decimal128` y se intentó un documento con importes `double`. Se comparó `explain("executionStats")` de la lectura por `id_pedido` con `EXPLAIN ANALYZE` de la reconstrucción relacional del pedido 1 (cinco tablas). La facturación por categoría se contrastó entre `$unwind` + `$group` y `GROUP BY`.

**Unidad 8.** `pruebas_sqlite.sql` demuestra `COMMIT`, `ROLLBACK` sin huérfanos, el catálogo offline (C1) y los `INSERT` inválidos (C7: `cantidad = 0` viola `CHECK`; `producto_id` inexistente viola la clave foránea). `simulador.py` sobre `foodstore_movil.db` ejecuta C2 a C6. Al terminar se volvió a consultar PostgreSQL: el conteo de `Pedido` y el stock de los productos usados no cambiaron.

## 3. Resultados

### Unidad 4 — planes y equivalencia

Volumen de la medición: 12 categorías, 20.003 clientes, 50.005 productos, 200.003 pedidos y 400.011 detalles. El día de la prueba aportó 10.198 pedidos y 20.396 detalles. Motor: PostgreSQL 16.15, `shared_buffers = 128MB`, `work_mem = 4MB`, dos workers.

| **Medición** | **Antes (join a `producto` y `categoria`)** | **Después (`detalle_pedido.categoria_nombre`)** |
|---|---|---|
| `actual time` del `Limit` | 71,667 .. 77,357 ms | 61,405 .. 66,748 ms |
| Mediana de 7 ejecuciones | 77,258 ms | 65,942 ms |
| Buffers del plan | shared hit=9424, read=2916 | shared hit=7905, read=2884 |
| Acceso a `detalle_pedido` | `Parallel Seq Scan` más dos hash join (producto y categoría) | `Parallel Seq Scan` y un solo hash join, contra `pedido` |

La mediana bajó 14,6 % (factor 1,17). El plan sigue haciendo sequential scan del detalle porque el filtro del día está en `pedido`; lo que se elimina es el join a `producto` y `categoria` para recuperar el nombre. La auditoría posterior a la migración devolvió 0 filas desincronizadas y `diferencias_antes_despues = 0`. Las nueve pruebas de anomalía cerraron con `auditoria=0` y `diferencias=0`, y el script terminó en `ROLLBACK`.

### Unidad 5 — simulacro

En el log anonimizado del PID 12492 la sesión duró 175 ms. Los cinco intentos fallidos y el login de `usuario_id = 202` quedaron registrados. `SELECT * FROM usuario` fue rechazado (`permiso denegado a la tabla usuario`). `SET ROLE admin_datos`, los dos `GRANT` y el `DELETE` también fueron rechazados. El cierre de sesión mostró `session_user = app_web`, `current_user = app_web` y `pg_has_role(..., 'admin_datos') = false`. No hubo escritura exitosa. La consulta de equivalencia de `usuario_anon` devolvió 0 filas de diferencia en ambos `EXCEPT ALL` (30 grupos, 242 usuarios).

### Unidad 6 — simulacro de desastre

Respaldo `foodstore_20261109_221751.dump`, formato custom, 8.433.690 bytes (8,04 MB), generado en 1,950 s el 2026-11-09 22:17:53 -03. El simulacro empezó a las 22:17:54.802 sobre `foodstore_restore_test`. Dictamen: aprobado.

| Métrica | Meta | Observado | Desglose |
|---|---|---|---|
| RPO | ≤ 24 h (respaldo diario) | 1,58 s | Antigüedad de ese dump en el ensayo. En producción el peor caso sigue siendo 24 h. |
| RTO | ≤ 300 s | 5,183 s | `createdb` 0,250 s + `pg_restore` 4,214 s + 3 validaciones 0,718 s |

Los conteos coincidieron al 100 %: categoría 12, cliente 20.003, producto 50.005, pedido 200.003, detalle 400.011. Productos inválidos: 0. Detalles inválidos: 0. El `VACUUM` con `INDEX_CLEANUP ON` bajó las tuplas muertas de `detalle_pedido` (23.974), `pedido` (7.719) y `producto` (3.450) a 0. El primer `VACUUM` sin esa opción había salteado el barrido de índices (1,63 % de páginas `LP_DEAD`, bajo el umbral del 2 %).

### Unidad 7 — documental contra relacional

El validador rechazó el documento con importes `double` (`1050.50`) con `MongoServerError` código 121: se esperaba `decimal` y llegó `double`. La suma `[0.1, 0.2]` en `double` dio `0.30000000000000004`; la misma suma en `Decimal128` dio `0.30`.

Lectura del pedido 1:

| Motor | Plan | Costo observado |
|---|---|---|
| MongoDB | `EXPRESS_IXSCAN` sobre `idx_pedidos_id_pedido_uq` | 1 clave, 1 documento, 3 ms |
| PostgreSQL | 4 `Nested Loop`, 11 `Index Searches`, 5 tablas | `shared hit=31`, execution time 0,063 ms |

Los totales por categoría coincidieron en los 7 rubros (Almacén 18.609,66; Postres 13.168,64; Panificados 12.721,24; Bebidas 12.470,99; Pizzas 3.600,00; Congelados 2.364,26; Snacks 811,44). Los filtros cruzados también: 4 pedidos `ENTREGADO` (ids 1, 2, 4 y 5) y 2 pedidos con `id_producto = 3` (ids 1 y 3).

### Unidad 8 — conflictos de sincronización

| Caso | Resultado medido |
|---|---|
| C2 | `CONFLICTO` (Muzzarella vista a 105.000 centavos, servidor a 120.000). Tras aceptar el precio, el reintento con otro UUID quedó en `SIMULADO_APROBADO`. |
| C3 | `RECHAZADO`. Napolitana: se pidieron 200 y el snapshot tenía 25. |
| C4 | `RECHAZADO`. Calzone Calabrés (id 7) con `eliminado = 't'`. La recarga deja la fila (lápida) y el catálogo visible devuelve 0 filas para ese id. |
| C5 | `SIMULADO_APROBADO` (Coca 1.5L y Agua Mineral, precio y stock alineados). |
| C6 | Reenviar `sync-uuid-c5` falla con `UNIQUE constraint failed: cola_sync.solicitud_uuid`. Repetir el semáforo no cambia el estado ya resuelto. |
| C7 | `cantidad = 0` viola `CHECK`. Un `producto_id` inexistente viola la clave foránea. No quedan filas. |

Después de la simulación, PostgreSQL seguía en 200.003 pedidos. El stock de Muzzarella (40), Napolitana (25), Coca 1.5L (100), Agua Mineral (150) y Calzone Calabrés (15) no se movió.

### Code review final (Unidad 8, sección 3.c)

| Hallazgo | Propuesta de la IA | Decisión | Comprobación |
|---|---|---|---|
| `fn_validar_stock_pedido` (TP2) | Sacar el `FOR UPDATE` para bajar la latencia | Descartada | Dos sesiones `READ COMMITTED` leyeron stock 2. Sin el bloqueo, la segunda vendió de más. Con `FOR UPDATE`, esperó, releyó 0 y lanzó «Stock insuficiente». |
| Índice parcial sobre `detalle_pedido (id_pedido) WHERE eliminado = FALSE` (TP5) | Crearlo para el borrado lógico | Descartada | `EXPLAIN (ANALYZE, BUFFERS)` del pedido 500 usó `pk_detalle_pedido` con 3 shared hit, con y sin el índice. En 5.000 filas con `pgbench` el índice extra subió la escritura 14,2 %. |
| `GRANT SELECT` de todas las tablas a un rol cajero (TP7) | Simplificar la facturación | Descartada | Ese `SELECT` alcanzaba `usuario.password_hash`. El esquema entregado no tiene ese rol ni ese grant. |
| Trigger de `categoria_nombre` (TP6) | Recalcular en cada `INSERT` de detalle | Corregida | Se condicionó con `WHEN (OLD.id_categoria IS DISTINCT FROM NEW.id_categoria)` para no reescribir la columna cuando la categoría no cambió. |

## 4. Decisiones de diseño

**FNBC aunque 3FN no la ve.** `deposito_id` es primo en `control_lote_almacen`, así que la dependencia del responsable hacia el depósito no rompe 3FN y sí rompe FNBC. Se descompuso igual. La vista de reunión natural conserva la consulta histórica.

**Desnormalizar solo el nombre de la categoría.** Copiar fecha o subtotal al detalle no acelera el filtro del día, que ya es sargable sobre `pedido.fecha_hora`, y agrega otra redundancia que hay que mantener. La columna se puede retirar sin perder cantidad ni precio.

**Mínimo privilegio por columna y por función.** No existe `rol_cajero`. `app_web` hereda solo lectura de catálogo y escritura del circuito de compra, sin `DELETE`, sin `WITH ADMIN OPTION` y sin `GRANT ALL`. Las anulaciones son bajas lógicas. `rol_logistica` y `rol_auditor` salen del dominio real (depósitos de la unidad 4 y revisión de seguridad), no de un catálogo genérico de roles.

**PostgreSQL sigue siendo la fuente de verdad.** MongoDB guarda una copia de lectura del pedido para traer el comprobante en un documento. No reemplaza el stock ni la venta concurrente: eso exige el `FOR UPDATE` de la unidad 2 y transacciones que cruzan entidades. SQLite manda solo sobre borradores locales. Un conflicto de precio se resuelve en la tablet (el cliente acepta o no); un quiebre de stock o una baja lógica se rechazan. La idempotencia del reenvío queda en la clave de `solicitud_uuid`.

**Contención del incidente, no la que propuso la IA.** `current_user = postgres` dentro de `fn_autenticar` es el efecto de `SECURITY DEFINER`, no una escalada. `usuario.rol = 'soporte'` no es el rol de motor `rol_soporte`, y `app_web` no es miembro de ese rol. No correspondía bajar el servicio ni restaurar un backup: el log con `log_statement = 'all'` muestra solo lecturas y errores de permiso. La contención aceptada es rotar la clave de `app_web`, cortar sus sesiones, limitar `pg_hba.conf` a la IP del backend, resetear la cuenta de aplicación `usuario_id = 202` y dejar de loguear el texto de `fn_autenticar`.

**RPO de laboratorio y RPO de operación.** El 1,58 s es la antigüedad del dump en el ensayo, porque se restauró un respaldo recién tomado. La meta operativa del `pg_dump` diario sigue siendo 24 h. Bajarla a menos de 5 min exige WAL/`PITR`, que quedó como mejora y no como mecanismo entregado. El RTO de 5,183 s cumple la meta de 300 s en esta base (unos 179 MB en disco, dump de 8 MB).

## 5. Herramientas de IA, bitácoras y protocolo

En las cinco unidades se usó Kiro para la especificación y OpenCode como asistente de terminal. Ningún script se ejecutó sin lectura previa. Las pruebas fueron sobre copia o dentro de una transacción con `ROLLBACK`. A la IA no se le enviaron datos personales: el log del simulacro se anonimizó (correos con `md5`, secretos redactados, IP de documentación RFC 5737) antes del prompt.

| Unidad | Dónde está la declaración | Qué se aceptó | Qué se descartó o se corrigió |
|---|---|---|---|
| 4 | `TP6_FNBC_Desnormalizacion/DUIA.md` y `kiro/` | La descomposición FNBC, la vista de reunión y `categoria_nombre` con triggers | Copiar fecha o subtotal al detalle |
| 5 | `TP7_Seguridad_Roles_Auditoria/BITACORA.md` (hace de DUIA; no hay un archivo aparte con ese nombre) y `docs/kiro_roles_spec/` | La matriz de seis roles más dos login, los `GRANT` por columna y la auditoría nativa | `DROP TABLE usuario CASCADE` (hay FK desde la unidad 4); funciones `SECURITY INVOKER`; `GRANT ALL` a `rol_app_escritura`; `SELECT` completo de `usuario` para soporte; `WITH ADMIN OPTION` en `app_web`; atrapar el error del simulacro en un `DO` (borra el `ERROR` del log); `md5(random())` para el anonimizado; apagar PostgreSQL y restaurar un backup por el `current_user = postgres` |
| 6 | `SPEC_U6.md` sección 4 e `informe_u6.md` sección 5. No hay un `DUIA.md` suelto | La estructura de `backup_foodstore.ps1`, el `Stopwatch` de `restore_test.ps1` y el esqueleto de `maintenance.sql` | Contraseñas o `PGPASSWORD` en los scripts (queda `pgpass.conf`); el filtro `precio` (la columna real es `precio_actual`); `VACUUM FULL`; el `VACUUM` que salteaba índices |
| 7 | `TP7_NoSQL_MongoDB/duia.md` | El agregado embebido, los índices y la arquitectura híbrida | Colección sin `$jsonSchema`; importes en `double`; «MongoDB no es ACID» y «siempre es más rápido que PostgreSQL» |
| 8 | `TP8_Movil_SQLite_Cierre/docs/DUIA.md`, `docs/bitacora_ia.md` y `docs/kiro/` | Esquema `STRICT`, semáforo rojo/amarillo/verde y los casos C1 a C7 | Cualquier camino en el que la tablet confirme la venta o descuente stock. Los cuatro hallazgos del code review de la sección 3 de este informe |

El code review pedido por la unidad 8 no es un archivo separado: está en `TP8_Movil_SQLite_Cierre/docs/informe.md`, sección 3.c, con cuatro hallazgos y la decisión humana de cada uno. Al menos dos quedaron descartados con una medición (`FOR UPDATE` y el índice parcial).

## Dónde está cada evidencia

| Unidad | Scripts | Specs Kiro | Declaración de IA | Evidencia |
|---|---|---|---|---|
| 4 | `tp_fnbc_control_lote.sql`, `tp_desnormalizacion_top_categorias.sql`, `herramientas/` | `kiro/requirements.md`, `kiro/design.md` | `DUIA.md` | `evidencias/plan_antes.txt`, `plan_despues.txt`, `mediciones_resumen.txt`, `pruebas_integridad.txt` |
| 5 | `sql/roles.sql`, `auditoria_test.sql`, `simulacro_incidente.sql`, `usuario_anon.sql` | `docs/kiro_roles_spec/` | `BITACORA.md` | `docs/informe_incidente.md`, `log_simulacro_anonimizado.txt`, `verificacion_equivalencia.md` |
| 6 | `backup_foodstore.ps1`, `restore_test.ps1`, `health_check.sql`, `maintenance.sql` | `SPEC_U6.md` | `informe_u6.md` §5 | `evidencias/resultado_simulacro_rto_rpo.txt`, `informe_u6.md` |
| 7 | `consultas_mongodb.js`, `consultas_postgresql.sql` | Cubierta en `duia.md` y el informe de la unidad | `duia.md` | `informe_nosql_mongodb.md` |
| 8 | `movil/schema_sqlite.sql`, `pruebas_sqlite.sql`, `semaforo.sql`, `simulador.py` | `docs/kiro/requirements.md`, `design.md`, `tasks.md` | `docs/DUIA.md`, `docs/bitacora_ia.md` | `docs/informe.md` §2 y §3.c, `movil/foodstore_movil.db` |